

Lane detection & Drivable Area Segmentation: Traditional Computer Vision Vs End-to-End Deep Learning Architectures

Vasileios Grapsopoulos 03791 Nikolaos Kalamaris 03768
Electrical & Computer Engineering, University of Thessaly
{vgrapsopoulos, nkalamaris}@uth.gr

Abstract

Two of the most critical and fundamental problems in autonomous driving are lane detection and the segmentation of the drivable area. The importance and complexity of these subjects are derived from the environmental variability and the fact that these systems have to remain robust under adverse scenarios to preserve safety. Due to the inherent differences between structural line detection and area-based segmentation, our system implements a dual-path pipeline that optimizes feature extraction specifically for each task. While modern CNN systems can provide more accurate results, deterministic classical computer vision techniques are often computationally faster, which is vital for real-time response. The objective of this study is to conduct an extensive review comparing the time response and potential vulnerabilities of our implementation vs the state-of-the-art DL architectures

1 Introduction

With the rapid growth of AI over the past few years, the world is constantly integrating automation into daily life. Autonomous driving, once a staple of science fiction, has recently seen real-world deployment in cities like Los Angeles with the success of driverless taxis. Although Deep Learning (DL) systems are primarily used in state-of-the-art implementations, the field remains rooted in foundational image processing and Computer Vision (CV) techniques.

Our work implements a traditional computer vision pipeline, leveraging divergent processing paths to tackle lane detection and drivable area segmentation due to the distinct nature of these two problems. Specifically, we utilize geometric edge-based analysis through Canny edge detection and the Hough transform for lane extraction, alongside color-space conversion and morphological operations for surface-based road segmentation. Finally, we compare our outcomes to modern Convolutional Neural Network (CNN) implementations to evaluate the relative merits and limitations of these two distinct approaches.

2 Related Work

The domain of autonomous driving perception has witnessed a paradigm shift from hand-crafted feature extraction to data-driven deep learning. This section reviews the evolution of these methodologies and situates our comparative study within the broader literature.

2.1 Traditional Feature-Based Approaches

Early research in lane detection relied heavily on geometric modeling and heuristic image processing. The foundational work by Canny et al. [2] established edge detection as a primary tool for structural

analysis. Subsequent approaches combined this with the Hough Transform [3] to identify linear lane markings, a method widely adopted due to its low computational overhead.

However, these deterministic algorithms suffer from significant limitations. As noted by Aly et al. [1], relying on fixed thresholds makes these systems fragile to lighting variations, shadows, and worn-out lane markings. Furthermore, the standard Hough Transform inherently assumes straight lines, requiring complex extensions like the Parabolic Hough Transform or Spline fitting to handle curvature—solutions that often introduce prohibitive computational costs for real-time systems. Our "Path A" implementation reflects these classical methodologies to establish a baseline for performance and failure mode analysis.

2.2 Deep Learning and Semantic Segmentation

The advent of Convolutional Neural Networks (CNNs) revolutionized the field by replacing manual feature engineering with learned representations. Approaches such as SCNN (Spatial CNN) [5] introduced mechanisms to capture the continuous, long, and thin structure of lanes, significantly outperforming geometric methods in complex scenarios.

For drivable area segmentation, the field moved towards fully convolutional architectures like FCN [4] and U-Net [6], which classify pixels semantically. While highly accurate, these early deep learning models were often single-task oriented. A vehicle would require separate heavy networks for object detection, lane finding, and road segmentation, creating a bottleneck for embedded inference.

2.3 Multi-Task Unified Architectures

To address the latency issues of running multiple networks, recent state-of-the-art research has pivoted toward multi-task learning. The YOLOP architecture proposed by Wu et al. [8] represents a significant milestone in this direction. By utilizing a shared encoder and a unified optimization objective, YOLOP performs detection and segmentation simultaneously without redundant feature extraction.

2.4 Distinction of Our Work

While existing literature focuses largely on proposing novel architectures or achieving state-of-the-art benchmarks on specific datasets, our work differs in its objective. We do not propose a new network layer; rather, we conduct a **comparative robustness analysis**. By implementing both a classical pipeline and the YOLOP architecture side-by-side, we aim to quantify the exact "cost" of deep learning in terms of latency versus the "gain" in geometric robustness.

3 Data

To highlight the impact of environmental variability, we created a small hybrid dataset by sampling the following:

1. TuSimple [here]
2. bdd100k [here]

Both of these datasets came from kaggle. TuSimple consists of highway pictures with clear vision of the road ahead and minimal light volatility. On the other hand bdd100k consists of city

road images with parked cars, traffic, trees, and different lighting. We selected images across a broad spectrum of difficulty to ensure our conclusions are more robust.

4 Methods

In this section, we present the two distinct methodologies employed for lane detection and drivable area segmentation. We first describe the traditional computer vision pipeline, which relies on geometric features, followed by the proposed end-to-end deep learning approach.

4.1 Traditional Computer Vision Pipeline.

The lane detection and drivable area segmentation system follows a modular architecture. The input is a raw image, which is processed through two distinct paths before being synthesized into a final output.

4.1.1 Path A: Lane Detection

This stage aims to extract structural boundaries using edge detection and linear modeling:

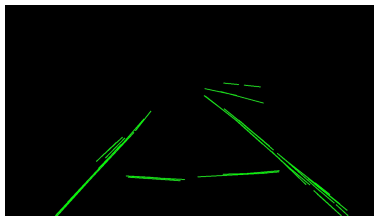
- **Canny Edge Detection:** The image is converted to grayscale and smoothed with a 5×5 Gaussian kernel. The Canny operator then identifies areas of high intensity gradients, producing a binary map of edges.
- **Region of Interest (ROI):** A trapezoidal mask is applied to the edge map to filter out irrelevant features such as the sky, trees, or side-road structures, focusing the algorithm on the ego-lane.
- **Hough Line Transform:** The system utilizes the Probabilistic Hough Transform to detect discrete line segments. This method maps edge pixels into the (ρ, θ) parameter space:

$$\rho = x \cos \theta + y \sin \theta \quad (1)$$

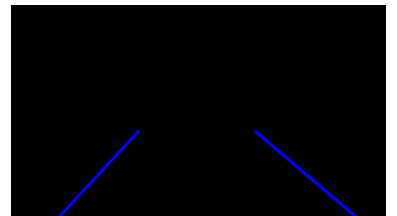
- **Slope-Based Averaging:** Detected segments are categorized by their slope. Segments with negative slopes are assigned to the left lane, while positive slopes are assigned to the right. A linear regression model fits these segments into two stabilized, continuous lines.



(a) Canny



(b) hough



(c) Final Lanes

Figure 1: Sequential stages of the drivable area segmentation pipeline.

4.1.2 Path B: Drivable Area Segmentation

The identification of the drivable road surface is performed through a color-based segmentation pipeline, which complements the geometric lane detection. This method focuses on isolating the asphalt area by analyzing the chromatic properties of the frame. The following techniques are employed:

- **Color Space Conversion:** The input image is converted from the BGR color space to HSV (Hue, Saturation, Value). This transformation is essential for decoupling luminance from chromatic information, providing robustness against varying lighting conditions and shadows on the road surface.
- **Color Thresholding:** A binary mask is generated by applying specific thresholds to the HSV channels. The range is tuned to isolate the neutral gray tones of the asphalt, typically targeting low saturation and mid-to-low brightness levels ($V \in [40, 180]$).
- **Morphological Operations:** To improve the consistency of the detected area, a morphological "Closing" operation is applied using a 7×7 kernel. This process fills internal gaps within the road surface (e.g., lane markings or cracks) and connects fragmented components into a single, solid region.
- **Spatial Constraint (ROI):** The resulting mask is constrained by a trapezoidal Region of Interest. This ensures that environmental noise, such as trees, buildings, or the sky, is excluded even if their color profile matches the road surface.
- **Alpha Blending Visualization:** The final segmentation is projected onto the original frame as a green semi-transparent overlay ($\alpha = 0.3$), allowing for a clear visual verification of the drivable path.

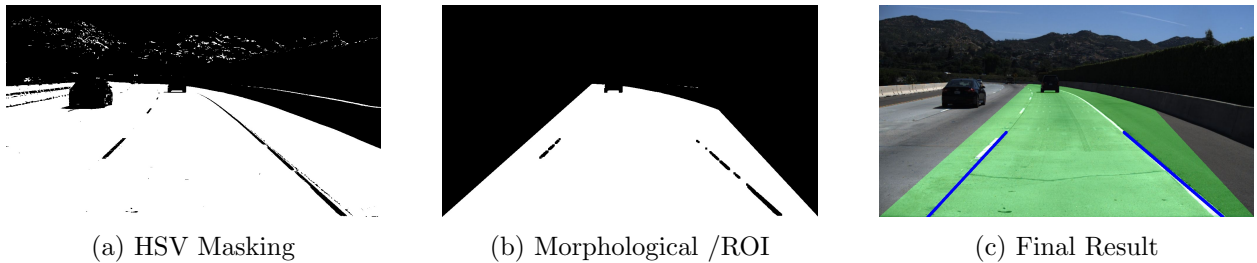


Figure 2: Sequential stages of the drivable area segmentation pipeline.

4.2 Deep Learning Approach: YOLOP

To overcome the inherent limitations of traditional computer vision methods—such as sensitivity to illumination changes, reliance on manual heuristic parameter tuning, and geometric constraints in curved lanes—this study employs the **YOLOP** architecture (You Only Look Once for Panoptic driving perception) [8]. Selected for its efficiency as a unified perception network, YOLOP processes the raw input image in a single forward pass to simultaneously perform three critical tasks: traffic object detection, drivable area segmentation, and lane line detection.

The core advantage of this multi-task learning paradigm is the elimination of computational redundancy. Instead of deploying separate models for each task, YOLOP utilizes a shared encoder

to extract features once, significantly lowering the inference latency which is vital for real-time autonomous driving applications.

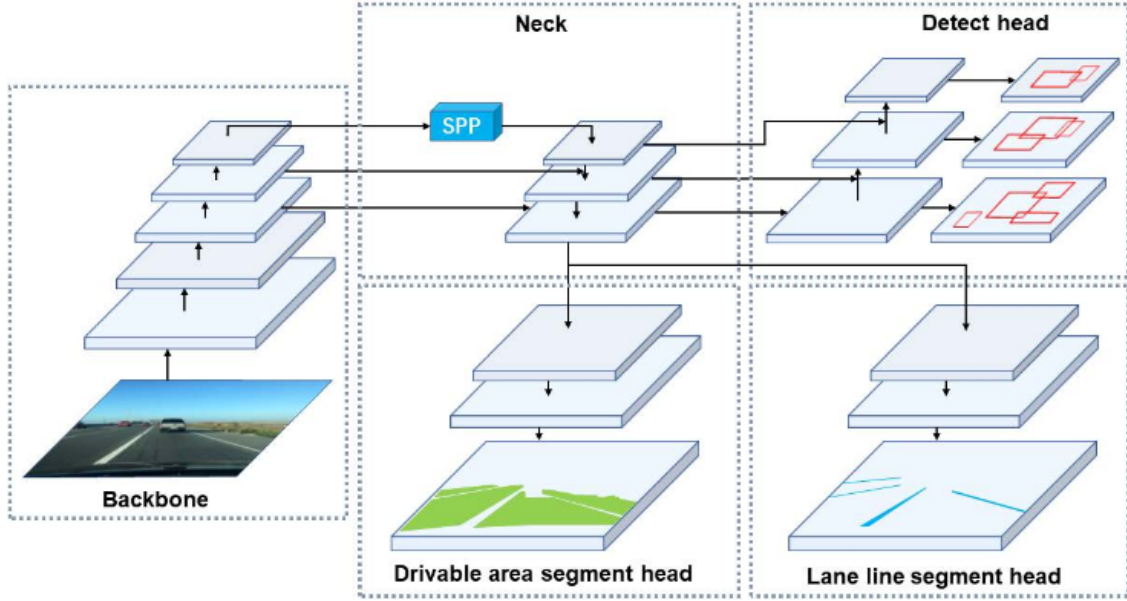


Figure 3: Overview of the YOLOP architecture. The input image passes through a shared encoder, and features are aggregated via the Neck before being distributed to three task-specific decoders. (Source: Wu et al. [8])

4.2.1 Network Architecture

As illustrated in Figure 3, the network adopts a lightweight encoder-decoder structure. The design philosophy prioritizes inference speed on embedded devices (such as the Jetson TX2) while maintaining high accuracy. The architecture comprises three main components:

Shared Encoder (Backbone): CSPDarknet The feature extraction module utilizes a **CSPDarknet** backbone, a derivative of the YOLOv4 architecture. This shared encoder is the computational engine of the system. To optimize the trade-off between computation and accuracy, it employs the **Cross Stage Partial (CSP)** strategy.

The CSP approach partitions the feature map of the base layer into two parts: one part propagates through a dense block of convolutions, while the other effectively bypasses the block and is concatenated at the end. This split-and-merge strategy solves the gradient duplication problem during optimization, allowing the network to learn richer gradient combinations while reducing the number of parameters and Floating Point Operations (FLOPs). This ensures that the visual features extracted are robust enough to support both detection and segmentation tasks simultaneously.

Feature Aggregation (Neck): SPP and FPN Successfully detecting objects at various scales—from large nearby vehicles to distant, thin lane markings—requires the fusion of features from different network depths. YOLOP incorporates a "Neck" structure to aggregate these features:

- **Spatial Pyramid Pooling (SPP):** This module pools features at different bin sizes to significantly increase the receptive field. It allows the network to capture global contextual

information, which is crucial for distinguishing drivable areas from the background.

- **Feature Pyramid Network (FPN):** The FPN fuses the strong semantic information found in deep layers with the precise spatial resolution of shallow layers. In YOLOP, features are fused using *concatenation*, ensuring that multi-scale semantic information is preserved before being passed to the decoders.

Task-Specific Decoders (Heads) The aggregated features are finally fed into three independent branches. The multi-task architecture is designed such that the computationally expensive feature extraction is shared, while the task-specific heads remain lightweight:

1. **Traffic Object Detection Head:** This branch employs an anchor-based multi-scale mechanism similar to YOLOv4. It predicts offsets for bounding boxes and object classification probabilities using a grid-based prediction mechanism.
2. **Drivable Area Segment Head:** This branch performs semantic segmentation to delineate the road surface. It accepts feature maps from the bottom layer of the FPN (size $(W/8, H/8, 256)$). Unlike heavy segmentation networks (e.g., PSPNet) that add extra SPP modules in the decoder, YOLOP keeps this branch simple to maximize speed. It utilizes three upsampling stages to restore the output to the original image resolution (W, H) .
3. **Lane Line Segment Head:** Most relevant to this study, this head outputs a pixel-wise segmentation mask for lane markings. Crucially, to reduce computational cost, the upsampling layers in both segmentation heads employ **Nearest Neighbor Interpolation** rather than complex deconvolution operations. This design choice highlights the focus on real-time performance.

Unlike the parametric line fitting of the Hough Transform used in our traditional pipeline, this pixel-level classification allows for the accurate mapping of complex lane topologies, including curves and merging lanes, without requiring explicit geometric modeling.

5 Experiments

To evaluate the effectiveness of the two proposed methodologies, we conducted a series of comparative experiments focusing on two key aspects: (1) **Inference Speed**, which is critical for real-time autonomous driving applications, and (2) **Detection Robustness**, specifically under challenging environmental conditions such as curved roads, shadows, and occlusion.

5.1 Experimental Setup

All experiments were conducted on the Google Colab cloud platform. The computational environment utilized an **Intel(R) Xeon(R) CPU @ 2.20GHz** (2 vCPUs) with approximately 13GB of RAM. Within this same environment, the deep learning inference (YOLOP) was accelerated by an **NVIDIA Tesla T4 GPU** with 16GB of VRAM.

5.2 Input Resolution and Pre-processing

It is important to clarify the input resolution handling. While the BDD100k and TuSimple datasets provide images at 1280×720 resolution, the YOLOP network architecture relies on a fixed input size for efficient batch processing.

As detailed in the original implementation experiments [8] and verified in the official source code [7], input images are resized to 640×640 pixels (or 640×384) using letterboxing to maintain the original aspect ratio without distortion. The network performs feature extraction and prediction at this reduced resolution. Finally, the output segmentation masks and bounding boxes are rescaled via interpolation to match the original 1280×720 resolution for visualization and metric evaluation.

5.3 Performance Analysis: Timing and Latency

One of the primary objectives of this study was to compare the computational cost of a deterministic algorithm versus a deep neural network. To evaluate computational efficiency, we performed inference on our test set comprising individual static images sampled from the BDD100k and TuSimple datasets.

We measured the average latency (in milliseconds) required to process a single frame. The Frames Per Second (FPS) metric was calculated as the theoretical throughput ($1/\text{Latency}$).

Table 1 presents the timing results. It is important to note the difference in workload: the Traditional CV pipeline performs Lane Detection and basic Drivable Area Segmentation (**DAS**). In contrast, the YOLOP architecture performs three tasks simultaneously: Lane Detection, DAS, and Traffic Object Detection (**OD**).

Table 1: Inference Speed Comparison

Method	Hardware	Tasks Performed	Latency (ms)	FPS
Traditional CV	CPU	Lanes + DAS	46.75 ms	21.39
YOLOP	CPU	Lanes + DAS + OD	586.9 ms	1.7
YOLOP	GPU-accelerated	Lanes + DAS + OD	61.7 ms	16.2

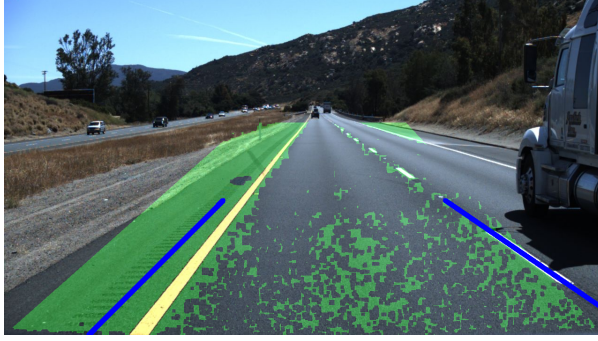
**Note: The traditional pipeline runs purely on the CPU. YOLOP results on GPU include the overhead for memory transfer between the host (CPU) and the device (GPU).*

5.4 Qualitative Comparison

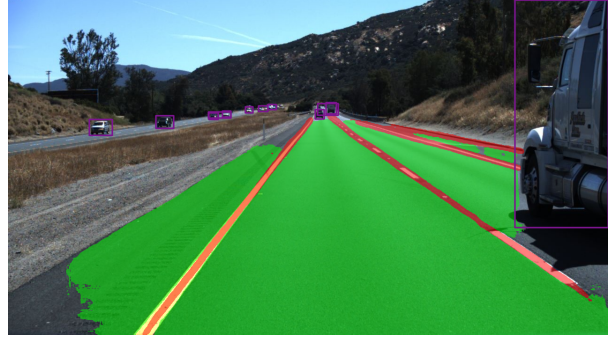
We evaluated both methods on the hybrid dataset constructed from TuSimple and BDD100k samples. The visual results highlight the fundamental differences between edge-based geometric modeling and semantic segmentation.

5.4.1 Scenario A: Straight Highways with Good Lighting

In ideal conditions (clear daylight, straight lanes), both methods perform adequately. The traditional pipeline successfully extracts the Region of Interest (ROI) and fits linear models to the lane markings but struggles to fit the drivable area due to the darker gray color of the asphalt. YOLOP produces a clean segmentation mask that aligns perfectly with the road boundaries.



(a) Traditional Pipeline



(b) YOLOP

Figure 4: Side-by-side comparison on straight highways with good lighting scenario.

5.4.2 Scenario B: Curved Roads and Complex Topologies

The limitation of the traditional pipeline is evident in curved scenarios, as shown in Figure 5. Constrained by the linear assumptions of the Hough Transform, the algorithm fails to follow the road curvature, resulting in straight lines that incorrectly intersect the drivable path. Conversely, YOLOP employs pixel-wise segmentation to dynamically map the road geometry, accurately capturing the curve (green/red masks) without rigid constraints.



(a) Traditional Pipeline



(b) YOLOP

Figure 5: Side-by-side comparison on a curved road scenario.

5.4.3 Scenario C: Shadows and Illumination Changes

Traditional computer vision pipelines rely heavily on fixed intensity and color thresholds, a dependency that leads to significant noise under complex lighting conditions. As observed in Figure 6(a), the algorithm incorrectly classifies sunlit buildings and parked vehicles as drivable area while simultaneously failing to detect parts of the road obscured by tree and buildings shadows. In contrast, YOLOP utilizes semantic feature extraction to robustly filter out these illumination variances, producing a coherent drivable area mask; however, despite this robustness to lighting, the model fails to detect the right lane boundary, likely due to the occlusion presented by the row of parked vehicles.



(a) Traditional Pipeline



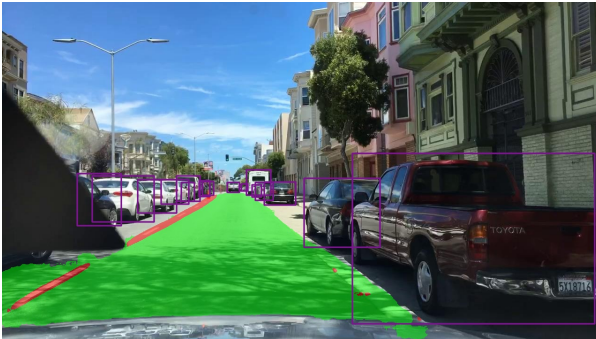
(b) YOLOP

Figure 6: Side-by-side comparison on shadows and illumination changes scenario.

5.5 Failure Mode Analysis

Despite its robustness, the YOLOP model is not infallible. As illustrated in Figure 7, we observed failure cases in specific scenarios:

- **Dense Urban Clutter:** In complex city environments, vehicles parked along the curbs often physically occlude the lane markers. As seen in Figure 7a, the continuous line of parked cars on the right results in a **complete failure to detect the right lane boundary**, as the model cannot infer the lane edge beneath the vehicles.
- **Worn-out Markings:** Extremely faded lane lines on rural roads (Figure 7b) sometimes result in missed detections. In these cases, the pixel intensity contrast is too low even for the deep network features, though it is worth noting that the drivable area segmentation usually remains correct.



(a) Dense Urban Clutter: Continuous parking along the curb occludes the markings, causing the model to completely miss the right lane boundary, detecting only fragments on the left.



(b) Worn-out Markings & False Positives: Faded road paint leads to fragmented center lane detection while detecting only half of the left boundary of the lane.

Figure 7: Visual analysis of YOLOP failure modes in challenging scenarios.

The traditional pipeline, however, fails catastrophically in night scenes where contrast is low (Figure 8) and the "Region of Interest" mask cannot easily exclude all distracting elements.



Figure 8: Side-by-side comparison in night scenes.

6 Conclusion

While traditional computer vision provides a strong foundation for image processing and often achieves lower latency than complex Deep Learning (DL) architectures, it lacks robustness in highly variable environments. Traditional CV implementations require extensive hyperparameter fine-tuning to account for specific weather conditions, varying asphalt textures, or line colors, and they remain highly sensitive to urban noise. Given more time, we would implement a Convolutional Neural Network (CNN) architecture trained on a larger dataset to achieve results that more closely approach state-of-the-art performance in diverse real-world conditions

References

- [1] Mohamed Aly. Real time detection of lane markers in urban streets. *arXiv preprint arXiv:1411.7113*, 2014.
- [2] John Canny. A computational approach to edge detection. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, pages 679–698, 1986.
- [3] Paul VC Hough. Method and means for recognizing complex patterns, 1962. US Patent 3,069,654.
- [4] Jonathan Long, Evan Shelhamer, and Trevor Darrell. Fully convolutional networks for semantic segmentation. *arXiv preprint arXiv:1411.4038*, 2014.
- [5] Xingang Pan, Jianping Shi, Ping Luo, Xiaogang Wang, and Xiaoou Tang. Spatial as deep: Spatial cnn for traffic scene understanding. *arXiv preprint arXiv:1712.06080*, 2017.
- [6] Olaf Ronneberger, Philipp Fischer, and Thomas Brox. U-net: Convolutional networks for biomedical image segmentation. *arXiv preprint arXiv:1505.04597*, 2015.
- [7] Han Wu et al. YOLOP: Official PyTorch implementation. <https://github.com/hustvl/YOLOP>, 2022.
- [8] Han Wu, Chengzhi Zhang, Yong Luo, Xiaolin Wang, Xinyu Li, Bohan Gao, Qingyong Shi, and Yimin Li. Yolop: You only look once for panoptic driving perception. *arXiv preprint arXiv:2108.11250*, 2022.